

Guia de Estudo

Skills de Desenvolvimento Assistido por IA
e os Documentos do Projeto (Specs & Plans)

Material baseado no fluxo de trabalho usado no projeto de Sistemas
Distribuídos (Sprints 01, 02, 2.1 e 03).

Conteúdo:

Resumo 1 — As skills: o que fazem, quando e como ativar.

Resumo 2 — Os artefatos criados: specs, plans, testes e estrutura.

Resumo 1 — As Skills (o quê, quando e como ativar)

O que é uma "skill"? É um guia de procedimento — um conjunto de instruções escritas que diz à IA *como conduzir* um tipo de tarefa, em vez de ela improvisar. Pense numa skill como uma metodologia de trabalho padronizada. Quando você pede para a IA "invocar uma skill", está pedindo que ela siga aquele método específico, com suas regras e etapas.

As quatro skills abaixo formam um fluxo encadeado: pensar → planejar → executar, tudo orquestrado por uma skill coordenadora.

As quatro skills do fluxo

1. brainstorming — "pensar antes de fazer"

O que faz: explora abordagens possíveis, compara vantagens e desvantagens (trade-offs) e leva às decisões estruturais e de design *antes* de qualquer código ser escrito.

Quando usar: no comecinho de cada sprint ou funcionalidade, sempre que houver uma decisão de arquitetura a tomar (ex.: usar broadcast ou multicast? AsyncIO ou Threads?).

Regra de ouro: existe um "portão" — nada de código até o design ser aprovado por você. Foi o que evitou, por exemplo, implementar o UDP errado antes de saber que o projeto rodaria em várias máquinas.

2. writing-plans — "escrever o passo a passo"

O que faz: depois do design aprovado, transforma a decisão em documentos. Quebra o trabalho em tarefas pequenas, cada uma com arquivos, código, comandos de teste e pontos de commit, em ordem.

Quando usar: logo após o brainstorm, para produzir a **spec** (o quê/por quê) e o **plan** (como/em que ordem).

Resultado: os arquivos de especificação e de plano (detalhados no Resumo 2).

3. executing-plans — "executar com disciplina"

O que faz: segue o plano tarefa por tarefa, na ordem, rodando os testes a cada passo, sem pular etapas.

Quando usar: quando o plano já existe e foi revisado, e você quer que a implementação realmente aconteça (foi quando você disse "executar").

Benefício: garante que cada item do plano vire código testado — foi assim que chegamos a 54/54 testes na Sprint 03.

4. using-superpowers — "o maestro"

O que faz: é a skill guarda-chuva, de orquestração. Decide *quando* usar cada uma das outras e em que ordem (brainstorm → plan → execução).

Quando usar: no início, para reger todo o processo de forma consistente. As outras três skills são "sub-habilidades" que ela coordena.

Observação: "superpowers" é apenas o nome do conjunto de skills no projeto — não é nada mágico, é a metodologia em si. (Às vezes aparece escrita como "using-powers".)

Como ativar as skills usando a IA

Você ativa uma skill simplesmente **pedindo em linguagem natural**, citando o nome dela. Não há comando mágico; o que importa é deixar claro qual método seguir e em que ordem. Exemplos reais do nosso projeto:

Pedido típico de início de sprint:

```
inicie a implementação invocando a skill brainstorm; writing-plans;
executing-plans; using-powers
```

Boas práticas ao acionar (o que funcionou bem):

- **Cite as skills na ordem do fluxo:** brainstorm primeiro, execução por último.
- **Forneça a fonte de requisitos:** anexar o PDF da sprint (visão geral, backlog, payloads, DoD, casos de teste) faz a IA extrair os dados oficiais em vez de adivinhar.
- **Aprove antes de avançar:** a skill de brainstorm vai parar e perguntar; responder às perguntas é o que libera a geração da spec e do plan.
- **Diga "executar" só quando estiver satisfeito** com a spec e o plan — é o gatilho da fase de execução.

Fluxo completo em uma frase

brainstorming (decide o design com você) → **writing-plans** (gera spec + plan) → **executing-plans** (implementa e testa) — tudo coordenado por **using-superpowers**.

Tabela-resumo

Skill	Para que serve	Quando usar
brainstorming	Explora opções e fecha o design (sem código ainda)	No início; em toda decisão de arquitetura
writing-plans	Gera a spec (o quê/por quê) e o plan (como/ordem)	Depois do design aprovado
executing-plans	Implementa o plano passo a passo, testando	Quando o plano está pronto e revisado
using-superpowers	Orquestra as demais na ordem certa	Durante todo o processo (maestro)

Resumo 2 — Os Artefatos Criados (specs, plans e mais)

Durante o trabalho, a IA produz e organiza vários tipos de arquivo. Cada um tem um papel distinto. Entender essa divisão ajuda a navegar o projeto e a estudar para a entrega.

A diferença central: spec x plan

Esta é a distinção mais importante de todas. Os dois documentos se complementam, mas respondem a perguntas diferentes:

SPEC (especificação) — responde "O QUÊ e POR QUÊ"

É o documento de **design**. Descreve o objetivo da sprint, as decisões tomadas e suas justificativas, os payloads oficiais, a arquitetura e as regras (parsing, timeouts, etc.).

Não contém o passo a passo de implementação — é a referência do *que* o sistema deve fazer e *por que* foi desenhado assim.

Exemplo no projeto: docs/superpowers/specs/2026-05-31-discovery-election-design.md

PLAN (plano) — responde "COMO e em que ORDEM"

É a **receita de execução**: a lista de tarefas numeradas, cada uma com os arquivos a criar/editar, o código, os comandos de teste e os pontos de commit.

É o documento que se segue na fase de execução; cada passo tem uma caixinha de marcar (checkbox) que vai sendo concluída.

Exemplo no projeto: docs/superpowers/plans/2026-05-31-discovery-election-implementation.md

A analogia que fixa a ideia

Construir uma casa: a **spec** é a *planta do arquiteto* (quantos cômodos, por que a cozinha fica ali, que material nas paredes — o "o quê" e o "por quê"). O **plan** é o *cronograma do mestre de obras* (primeiro a fundação, depois as colunas, depois a alvenaria — o "como" e a "ordem"). E o **código** é a casa construída.

Os demais artefatos do projeto

Código-fonte (a implementação)

Os arquivos `.py` que realmente fazem o sistema funcionar (ex.: `messaging.py`, `master_async.py`, `worker_async.py`, `discovery.py`, `m2m.py`). É a "casa construída" a partir da spec e do plan.

Testes (a prova de que funciona)

Arquivos na pasta `tests/` que verificam automaticamente cada comportamento (ex.: os casos CT01–CT09 do PDF). Cada decisão da spec vira um teste no plan, e cada teste comprova um pedaço do sistema.

Há também o `run_tests.py`: um executor de testes que roda tudo sem precisar instalar nada extra.

Scripts de demonstração

Arquivos em `scripts/` (ex.: `run_negociacao_demo.py`) que executam o fluxo completo de ponta a ponta, úteis para ver o sistema funcionando e para apresentar ao professor.

README (o manual da sprint)

Um arquivo por sprint explicando os payloads, como rodar, e o checklist da "Definição de Pronto" (DoD). É o primeiro lugar para entender o que aquela entrega faz.

Como specs/plans e código se acumulam

Uma diferença importante de comportamento, que combinamos no projeto:

- **Specs e plans acumulam por adição:** cada sprint ganha o seu próprio par de documentos (spec + plan), enxutos e focados, guardados juntos em `docs/`. Eles não incham — um novo se referencia ao anterior.
- **Código acumula por cópia:** cada pasta de sprint (ex.: `sprints/sprint03_negociacao/`) é um retrato completo e executável do projeto naquele ponto, contendo tudo que veio antes mais as novidades daquela sprint.

Rastreabilidade: o elo entre tudo

O processo é sólido porque cada peça se liga à seguinte: uma **decisão** na spec vira um **passo verificável** no plan, que vira **código** acompanhado de um **teste** que o comprova. Assim nada se perde, e é possível auditar do início ao fim.

Tabela-resumo dos artefatos

Artefato	Pergunta que responde / papel	Onde fica
spec	O QUÊ e POR QUÊ (design e decisões)	<code>docs/superpowers/specs/</code>

plan	COMO e em que ORDEM (passo a passo)	docs/superpowers/plans/
código (.py)	A implementação que executa	sprints/.../src/
testes	Provam que o código atende à spec	sprints/.../tests/
script de demo	Mostra o fluxo completo rodando	sprints/.../scripts/
README	Manual da sprint + checklist (DoD)	raiz de cada sprint

Guia de estudo gerado para o projeto de Sistemas Distribuídos — fluxo de desenvolvimento assistido por IA. Bons estudos!